Arbeitsblatt Level 16 · Welt D · Sprachdesign & Ausblick · Niveau: Zusatz (über das KC hinaus)

Ausblick: Die Turingmaschine

Aufgabenbörse · Band, Kopf und Übergänge · Church-Turing-These und Halteproblem · Abschluss der Einheit

Name

Datum

Kernfrage *Was kann eine Maschine, die auf ihr Band schreiben darf – und gibt es Grenzen, die keine Maschine überwindet?*

Letztes Level! Zuerst löst ihr in der **Aufgabenbörse** die Aufgabe eines anderen Teams. Dann lernst du die mächtigste Maschine dieser Einheit kennen: die **Turingmaschine** – und erfährst, dass selbst sie Grenzen hat.

1 Aufgabenbörse

Basis AFB I–III

Ihr erhaltet die Aufgabe von Team _____ (Kurzversion). **Löst** sie im Team in 20 Minuten auf einem eigenen Blatt – wie im Abitur: Operatoren beachten, Notation einhalten. Vergleicht danach mit dem Erwartungshorizont des Autorenteams.

a) **Gib** dem Autorenteam eine Rückmeldung: Was war klar formuliert? Wo fehlten Angaben (Σ, Hinweis zum Fehlerzustand, BE)?

.....

.....

Warum der Kellerautomat an $a^n b^n c^n$ scheitert

Für $L = \{a^n b^n \mid n \geq 1\}$ legt der Kellerautomat für jedes a ein A auf den Keller und nimmt für jedes b eines herunter. Der Keller ist ein **Zähler, der beim Vergleichen verbraucht wird**: Nach den b ist er leer – die Information „wie viele a?“ ist weg, für die c gibt es nichts mehr zu vergleichen. Auch Tricks (zwei A pro a) helfen nicht: Beim ersten b weiß der Automat nicht, wann die b aufhören. Anschaulich: **Ein Keller kann nur einmal abgezählt werden** (ein strenger Beweis braucht mehr Theorie). Eine Maschine, die ihre Eingabe **verändern** und beliebig oft **hin- und herlaufen** darf, hat dieses Problem nicht.

2 Die Turingmaschine liest, schreibt und bewegt sich

Basis AFB I–II

Eine **Turingmaschine (TM)** hat ein beidseitig unbegrenztes **Band** aus Feldern, einen **Schreib-Lese-Kopf** und endlich viele Zustände. Leere Felder enthalten das **Blank** ␣. In jedem Schritt liest der Kopf das Zeichen unter sich und führt genau einen Übergang aus:

(Zustand, gelesenes Zeichen) → (Folgezustand, geschriebenes Zeichen, Bewegung L / R / N)

Im Zustandsgraphen steht an der Kante „gelesen / geschrieben, Bewegung“, z. B. „1 / 0, L“. Gibt es keinen passenden Übergang, **hält** die Maschine. Hält sie in einem Endzustand, ist die Eingabe akzeptiert; der Bandinhalt ist die Ausgabe.

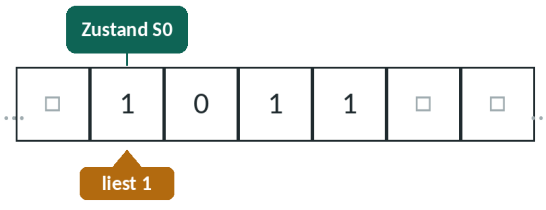


Abbildung 1: Band, Kopf und Zustand – die Maschine liest gerade die 1 am Anfang von 1011

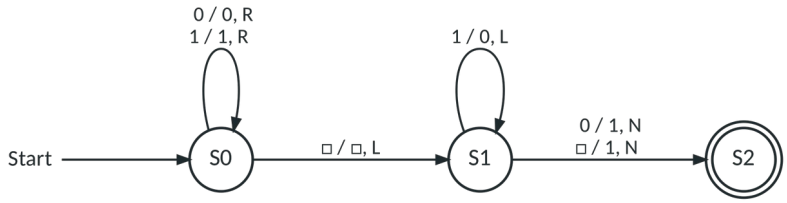


Abbildung 2: Turingmaschine „Binärzahl + 1“, Σ = {0, 1}. Der Kopf startet auf der ersten Ziffer.

a) **Beschreibe** die Bedeutung der Zustände S0, S1 und S2: Was tut die Maschine jeweils?

b) **Stelle** die Verarbeitung der Eingabe `1011` als Konfigurationsfolge **dar**. Notiere pro Schritt Zustand, Bandinhalt und Kopfposition (in eckigen Klammern).

Schritt	Zustand	Band (Kopfposition in eckigen Klammern)	Übergang
0	S0	□ [1] 0 1 1 □ □	–
1	S0	□ 1 [0] 1 1 □ □	1 / 1, R

c) **Gib an**, was die Maschine bei der Eingabe `111` ausgibt, und **erläutere**, warum sie dafür ein Feld links vom ersten Zeichen beschreibt.

3 Unplugged: Du bist der Schreib-Lese-Kopf Standard AFB II

Partnerarbeit mit dem Papierstreifen-Band und der Regelkarte (von der Lehrkraft). Eine Person ist der **Kopf**: Sie schiebt die Kopf-Marke, liest vor, radiert und schreibt. Die andere Person ist die **Steuerung**: Sie hält die Zustandskarte hoch und sagt nach jeder Regel den neuen Zustand an. Nach jedem Schritt wird getauscht.

a) **Führe** die Maschine „Binärzahl + 1“ für die Eingabe `1101` **aus**. Zähle die Schritte und notiere das Ergebnis. **Prüfe** es durch Umrechnen ins Dezimalsystem.

b) **Entwickle** eine Turingmaschine über Σ = {a, b}, die jedes a durch b ersetzt und am Ende der Eingabe hält. Teste sie auf dem Papierband mit `abab`.

Platz für deinen Zustandsgraphen

4 $a^n b^n c^n$ – was kein Kellerautomat schafft Standard AFB II

Die Turingmaschine in Abbildung 3 erkennt $L = \{a^n b^n c^n \mid n \geq 1\}$. Sie darf das Band verändern und beliebig oft hin- und herlaufen.

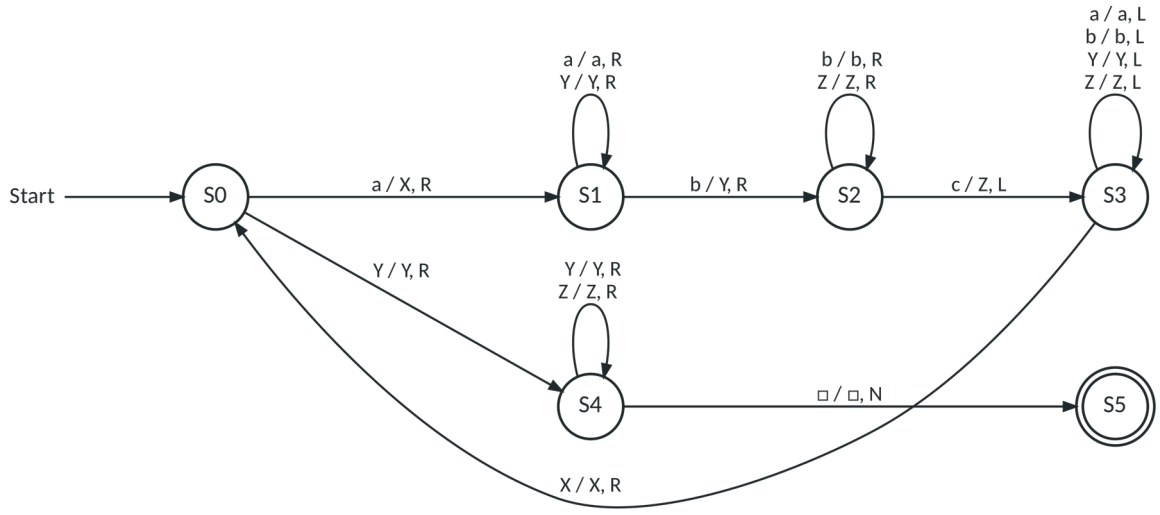


Abbildung 3: Turingmaschine für $a^n b^n c^n$, Σ = {a, b, c}, Bandalphabet Γ = {a, b, c, X, Y, Z, □}

a) **Erläutere** die Strategie der Maschine: Wozu dienen die Markierungen X, Y, Z, und was passiert in den Zuständen S0 bis S4?

b) **Gib** für die Eingabe `aabbcc` den Bandinhalt und den Zustand nach der ersten Runde **an** (eine Runde endet, wenn die Maschine in S3 ein X liest und nach S0 wechselt).

c) **Begründe** anhand der Eingabe `aabbc`, dass die Maschine diese Eingabe ablehnt: In welchem Zustand hält sie, und warum?

AutomatenLab Turing-Labor

Schritt für Schritt durch „Binärzahl + 1“ und „$a^n b^n c^n$“ laufen (programm=binaer-plus-1 | anbncn), Band beobachten, eigene Eingaben testen. Challenge 1: fehlende Regel ergänzen.

Link: https://automatenlab.frankjuchim.de/#/turing?programm=anbncn

5 Rätsel: Der fleißige Biber Profi AFB II–III

Zwei Maschinen mit den Zuständen A, B und dem Haltezustand H starten auf einem **leeren Band**. Welche schreibt mehr Einsen, bevor sie hält? (Σ = {1}; Zustandsnamen wie in der Fachliteratur.)

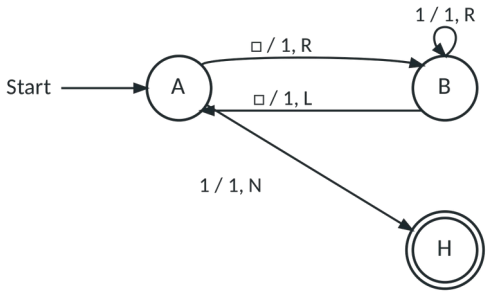


Abbildung 4: Maschine K

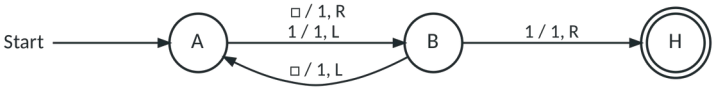


Abbildung 5: Maschine B

a) **Bestimme** für beide Maschinen die Anzahl der Einsen auf dem Band und die Anzahl der Schritte bis zum Halten.

b) Der **fleißige Biber** mit n Zuständen ist die Maschine, die unter allen haltenden Maschinen mit n Zuständen (plus H) die meisten Einsen schreibt. **Beurteile**, ob es eine Maschine mit zwei Zuständen gibt, die mehr Einsen schreibt als Maschine B. Recherchiere, was über den fleißigen Biber mit 5 und 6 Zuständen bekannt ist.

6 Unärer Addierer Profi AFB III

Zahlen werden als Strichlisten geschrieben: 3 + 2 ist die Eingabe `111+11`. **Entwickle** eine Turingmaschine über Σ = {1, +}, die die Summe als Strichliste ausgibt (hier `11111`). Idee: Das + durch eine 1 ersetzen, ans Ende laufen und dort eine 1 löschen. Selbstkontrolle: Turing-Labor, Challenge 2.

Platz für deinen Zustandsgraphen

7 Zwei große Ideen: Grenzen jeder Maschine Standard AFB III

Church-Turing-These: Alles, was sich überhaupt mit einem Algorithmus berechnen lässt, kann eine Turingmaschine berechnen. Jeder Computer, jede Programmiersprache ist genau so mächtig wie dieses einfache Modell – nicht mächtiger. (Eine These, kein Satz: Sie lässt sich nicht beweisen, hat sich aber seit 1936 in jedem Fall bestätigt.)

Das Halteproblem – eine Geschichte: Eine Firma verspricht das Programm HALT: Man gibt ihm ein beliebiges Programm P und eine Eingabe, und HALT sagt nach endlicher Zeit korrekt „hält“ oder „läuft ewig“. Eine Informatikerin baut daraus

das Programm TROTZ: Es fragt HALT, ob das Programm TROTZ selbst (mit der eigenen Beschreibung als Eingabe) hält – und tut dann **das Gegenteil**: Sagt HALT „hält", läuft TROTZ in eine Endlosschleife; sagt HALT „läuft ewig", hält TROTZ sofort. Egal, was HALT antwortet, es irrt sich. Also kann HALT nicht existieren. Alan Turing zeigte 1936: **Kein Algorithmus kann für alle Programme entscheiden, ob sie halten.** Es gibt Grenzen, die keine Maschine überwindet.

a) **Erläutere** in eigenen Worten, warum das Programm TROTZ das Programm HALT in jedem Fall widerlegt.

.....

.....

b) **Beurteile** die Aussage: „Mit einem schnelleren Rechner ist das Halteproblem irgendwann lösbar."

.....

.....

Abschluss der Einheit

AutomatenLab **Level-Quiz Welt D**

Abschluss-Quiz zu Sprachdesign und Turingmaschine (#/quiz?welt=D) – oder das Mini-Exit-Ticket zu Level 16 (#/quiz?ds=16).

Link: https://automatenlab.frankjuchim.de/#/quiz?welt=D

Lernlandkarte-Selbstcheck

Nimm deine **Lernlandkarte** (Kompetenzraster „Ich kann …") zur Hand und markiere für jedes Level: sicher / unsicher / noch üben. Für jedes „noch üben" notierst du eine konkrete Aufgabe (Merkblatt, Boss-Level, App-Modul), mit der du es nachholst – das ist dein Plan für die Übungsklausur.

Feedback zur Einheit

Das hat mir beim Verstehen am meisten geholfen (Rollenspiel, App, Boss-Level, Merkblätter, Projekt …):	Das hätte ich mir anders gewünscht:	Eine Frage, die mir noch offen ist:

Selbstcheck – Level 16 geschafft, wenn …

- ☐ ich den Aufbau einer Turingmaschine (Band, Kopf, Zustände, Blank) und die Form eines Übergangs angeben kann.
- ☐ ich eine kleine Turingmaschine Schritt für Schritt ausführen und die Konfigurationsfolge darstellen kann.
- ☐ ich anschaulich erklären kann, warum ein Kellerautomat an $a^n b^n c^n$ scheitert und eine Turingmaschine nicht.
- ☐ ich die Church-Turing-These und die Idee des Halteproblems in eigenen Worten wiedergeben kann.